مكتب

Reference Notes

Week 4: String Basics

7th Grade Computer Science - Module 2

1 Introduction

1.1 Unlocking the Power of Text

How do different data structures help us organize and manipulate information? So far, you've worked with numbers and simple text in print statements. But what if you want to analyze messages, create word games, or process usernames? This week, we discover strings - Python's powerful way to work with text data!

Prerequisites

Before starting this week, you should be comfortable with:

- Variables and basic data types (Module 1, Week 2)
- Using `input()` to get text from users (Module 1, Week 4)
- Working with `for` loops (Module 1, Week 6)
- Basic comparison operators (Module 1, Week 7)

1.2 What You Already Know

You've been using strings all along! Every time you wrote `print("Hello")`, used `input()`, or stored someone's name in a variable, you were working with strings. Now we'll explore how to manipulate, analyze, and transform text like a pro!

1.3 What You'll Be Able to Do

By the end of this week, you'll:

- Access individual characters in any text using indexing *(like finding the 3rd letter of a name)*
- Extract portions of text using slicing *(like getting the first 5 characters of a password)*
- Use built-in string methods to transform text *(like converting to uppercase or finding substrings)*
- Choose the right string operation for different text-processing tasks
- Build programs that manipulate and analyze text data

2 VIDEO 1: Understanding Strings

2.1 What Are Strings?

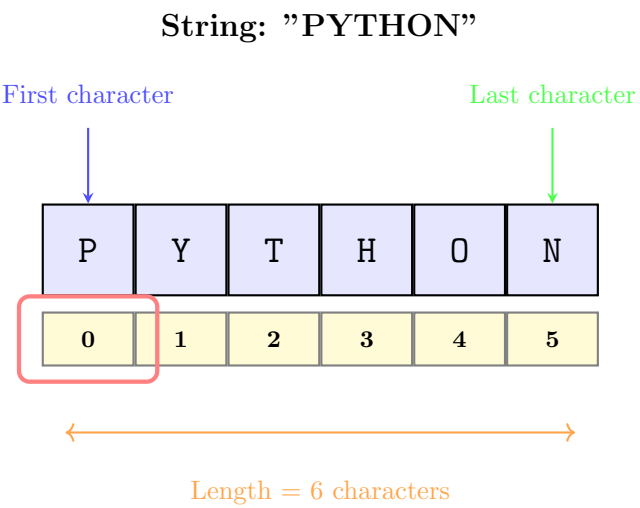
A string is a sequence of characters - letters, numbers, symbols, even spaces! In Python, we create strings using quotes. You can use single quotes `'like this'` or double quotes `"like this"` - Python treats them the same way. Strings let us work with text data in powerful ways!

2.2 Creating and Using Strings

```python
# Different ways to create strings
name = "Ahmed"
message = 'Welcome to Python!'
empty = ""  # Empty string (zero characters)
number_text = "123"  # These are characters, not a number!

# Strings can span multiple lines with triple quotes
story = """Once upon a time,
in a land far away,
there lived a programmer."""
```

2.3 Visualizing Strings as Sequences



Important: Indexing starts at 0, not 1!
First character is at index 0, last is at index 5

2.4 String Operations You Already Know

```python
# Concatenation (joining strings)
first = "Hello"
second = "World"
combined = first + " " + second  # "Hello World"

# Repetition
laugh = "Ha" * 3  # "HaHaHa"

# Length
text = "Python"
```

```
size = len(text)  # 6

# Checking membership
if "fun" in "Python is fun!":
    print("Found it!")
```

Tip

Remember: `len()` gives you the total number of characters. The last valid index is always `len(text) - 1` because indexing starts at 0!

Common Error

Common Mistake: Thinking strings of numbers are actual numbers

```
num_string = "123"
result = num_string + 5  # ERROR! Can't add string to number
# Fix: Convert string to integer first
result = int(num_string) + 5  # 128
```

Quick Check

What will this code print?

```
word = "Hello"
print(len(word))
print(word * 2)
print("e" in word)
```

3 VIDEO 2: String Indexing

3.1 Understanding String Indexing

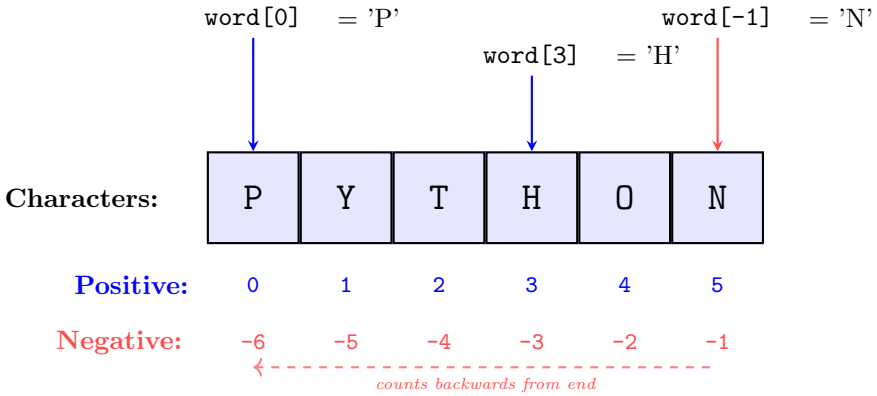
Every character in a string has a position number called an index. Python starts counting from 0 (not 1!). We use square brackets `[]` to access characters at specific positions. This lets us examine text character by character!

3.2 Accessing Individual Characters

```
text = "PYTHON"
# Positive indexing (from start)
first = text[0]   # 'P'
second = text[1]  # 'Y'
last = text[5]    # 'N'

# Negative indexing (from end)
last_char = text[-1]   # 'N' (last character)
second_last = text[-2] # 'O'
```

3.3 Visualizing Positive and Negative Indices



Positive indices count forward from 0
Negative indices count backward from -1

3.4 Working with String Indices

```
name = "Sarah"
# Loop through string by index
for i in range(len(name)):
    print(f"Index {i}: {name[i]}")

# Check first and last characters
password = input("Enter password: ")
if password[0] == password[-1]:
    print("Password starts and ends with same character!")
```

3.5 Index Boundary Checking

```
word = "Hi"
print(word[0])  # 'H' - Valid
print(word[1])  # 'i' - Valid
# print(word[2])  # ERROR! IndexError
# print(word[5])  # ERROR! IndexError

# Safe indexing with checking
index = 3
if index < len(word):
    print(word[index])
else:
    print("Index out of range!")
```

Rule

String Indexing Rules:

- Valid positive indices: 0 to `len(string) - 1`
- Valid negative indices: -1 to `-len(string)`
- Out of range index = IndexError!

Common Error

Common Mistake: Starting index at 1 instead of 0

```
name = "Ali"
# Wrong: Thinking first character is at index 1
first = name[1]  # This gives 'l', not 'A'!
# Right: First character is always at index 0
first = name[0]  # 'A'
```

Quick Check

Given `text = "COMPUTER"`, what do these expressions return?

- `text[3]`
- `text[-1]`
- `text[0] + text[-1]`

4 VIDEO 3: String Slicing

4.1 Understanding String Slicing

Slicing lets you extract a portion of a string - like cutting out part of a sentence. You specify a start position, an end position, and optionally a step size. The syntax is `string[start:end:step]`. This is incredibly powerful for text processing!

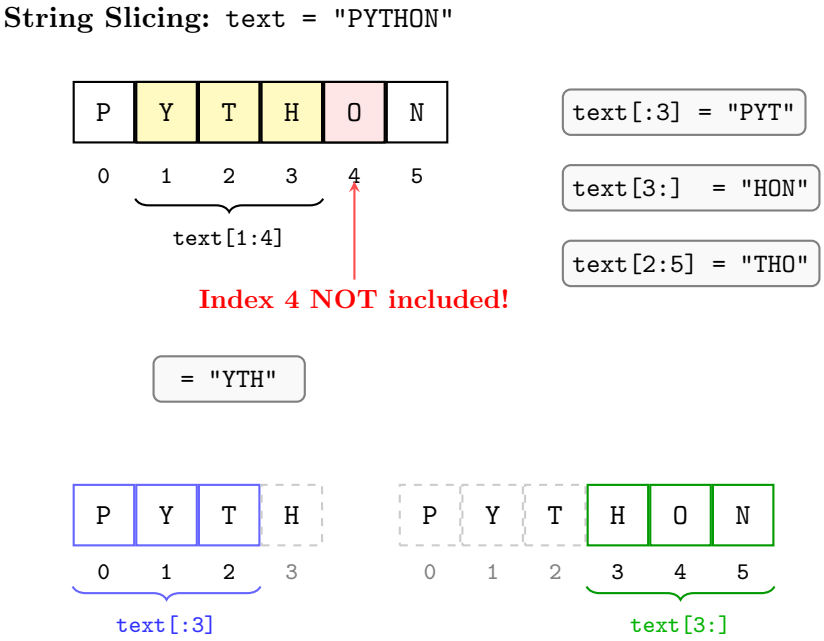
4.2 Basic Slicing Operations

```
text = "PYTHON"
# Basic slicing [start:end]
first_three = text[0:3]    # "PYT" (indices 0,1,2)
middle = text[2:4]         # "TH" (indices 2,3)

# End index is EXCLUSIVE!
piece = text[1:4]  # "YTH" (gets indices 1,2,3 NOT 4)

# Omitting start or end
from_start = text[:3]  # "PYT" (same as text[0:3])
to_end = text[3:]      # "HON" (from index 3 to end)
copy = text[:]         # "PYTHON" (entire string)
```

4.3 Visualizing String Slicing



Remember: `[start:end]` means "from start, up to (but NOT including) end"

4.4 Advanced Slicing with Step

```
message = "HELLO WORLD"
# Using step parameter
every_other = message[::2]     # "HLOWRD" (every 2nd char)
reversed_text = message[::-1] # "DLROW OLLEH" (backwards!)

# Combining start, end, and step
portion = message[0:5:2]  # "HLO" (0,2,4 from "HELLO")

# Negative indices in slicing
last_three = message[-3:]  # "RLD"
without_last = message[:-1] # "HELLO WORL"
```

4.5 Common Slicing Patterns

Pattern	Code	Description
First n characters	`text[:n]`	Start to position n
Last n characters	`text[-n:]`	Last n to end
Remove first character	`text[1:]`	Skip first
Remove last character	`text[:-1]`	Skip last
Reverse string	`text[::-1]`	Backwards
Every nth character	`text[::n]`	Step by n

Tip

Slicing is Forgiving! Unlike indexing, slicing doesn't give errors for out-of-range indices:

```
word = "Hi"
print(word[0:10])  # "Hi" (no error, just stops at end)
print(word[5:10])  # "" (empty string, no error)
```

Quick Check

Given `name = "PAKISTAN"`, what do these slices return?

- `name[2:5]`
- `name[:4]`
- `name[-4:]`
- `name[::2]`

5 VIDEO 4: String Methods

5.1 Understanding String Methods

Methods are built-in functions that strings have. You call them using dot notation: `string.method()`. Python provides dozens of methods for common text operations - no need to write these yourself!

5.2 Case Conversion Methods

```
text = "Hello World"
# Change case
upper = text.upper()       # "HELLO WORLD"
lower = text.lower()       # "hello world"
capital = text.capitalize() # "Hello world"
title = text.title()        # "Hello World"

# Original string unchanged!
print(text)  # Still "Hello World"
```

5.3 Searching and Testing Methods

```
email = "user@example.com"
# Finding substrings
at_pos = email.find("@")      # 4 (index of @)
dot_pos = email.find(".")     # 12 (index of .)
not_found = email.find("xyz") # -1 (not found)

# Testing string content
name = "Ahmed123"
print(name.isalpha())   # False (has numbers)
print("Ahmed".isalpha()) # True (only letters)
print("123".isdigit())   # True (only digits)
print(" ".isspace())     # True (only whitespace)
```

5.4 Text Cleaning and Transformation

You can also clean and modify text easily. For example, removing extra spaces, replacing words, or counting occurrences:

```
# Removing whitespace
messy = "  Hello World  \n"
clean = messy.strip()      # "Hello World" (no spaces/newlines)

# Replacing text
message = "I love cats and cats are great"
new_msg = message.replace("cats", "dogs")
# "I love dogs and dogs are great"

# Counting occurrences
text = "the quick brown fox jumps over the lazy dog"
count_the = text.count("the")  # 2
```

Rule

Method Syntax Pattern: `result = string.method(arguments)`
Examples:

- `text.upper()` - no arguments needed
- `text.find("abc")` - one argument
- `text.replace("old", "new")` - two arguments

Common Error

Common Mistake: Forgetting that methods return NEW strings

```
name = "ahmed"
name.upper()  # This doesn't change 'name'!
print(name)   # Still "ahmed"

# Correct: Store the result
name = name.upper()  # Now name is "AHMED"
```

Quick Check

What will this code output?

```
word = "Python Programming"
print(word.lower().count("p"))
print(word.find("gram"))
print(word[:6].upper())
```

6 VIDEO 5: Summary

6.1 What We Learned This Week

You've unlocked the power of text manipulation in Python! You can now access any character in a string, extract portions of text, and use powerful built-in methods to transform and analyze strings.

These skills are essential for processing user input, analyzing data, and building text-based applications!

6.2 String Operations Toolkit

Operation	Syntax	Example
Get length	`len(string)`	`len("Hi")` → 2
Access character	`string[index]`	`"Cat"[0]` → 'C'
Extract portion	`string[start:end]`	`"Hello"[1:4]` → "ell"
Reverse string	`string[::-1]`	`"ABC"[::-1]` → "CBA"
Convert case	`string.upper()/lower()`	`"hi".upper()` → "HI"
Find substring	`string.find(sub)`	`"Hello".find("lo")` → 3
Replace text	`string.replace(old,new)`	Replace parts of string

★ Landmark Moment

You now understand how Python organizes text as sequences of characters! With indexing and slicing, you can precisely access any part of a string. With methods, you can transform text without writing complex loops. You're ready to build word games, analyze text, and process user input like a pro!

7 Quick Reference

Quick Reference

Week 4 Essential Patterns:
String Indexing:

- Pattern: `char = string[index]`
- Common use: Accessing first/last character
- Common mistake: Starting at 1 instead of 0

String Slicing:

- Pattern: `substring = string[start:end:step]`
- Common use: Extracting parts of text
- Debug tip: Remember end index is exclusive

String Methods:

- Pattern: `result = string.method(args)`
- Common use: Case conversion, finding, replacing
- Common mistake: Forgetting methods return new strings

Safe String Access:

- Pattern: `if index < len(string):  string[index]`
- Common use: Avoiding IndexError
- Debug tip: Use slicing for safer extraction (no errors)

7.1 Reflection Questions

1. Why do you think Python starts counting indices at 0 instead of 1? What advantages might this have?
2. When would you use negative indexing instead of positive indexing? Give a specific example.
3. What's the difference between `string[3]` and `string[3:4]`? When would you use each?
4. Think of your favorite app or website. What string operations might it use for usernames and passwords?
5. If you wanted to check if a word is a palindrome (reads same forwards and backwards), which string operation would help most?

Next week: We explore string immutability - why strings can't change and how Python handles text in memory. Get ready for some mind-bending concepts!